# LAPORAN TUGAS KECIL 2
# IF2211 STRATEGI ALGORITMA

# VOXELIZATION OBJEK 3D MENGGUNAKAN OCTREE DENGAN ALGORITMA DIVIDE AND CONQUER

Disusun oleh:
Reva Natania Sitohang - 13524098
Amanda Aurellia Salsabilla - 13524131


**PROGRAM STUDI TEKNIK INFORMATIKA**
**SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA**
**INSTITUT TEKNOLOGI BANDUNG**
**BANDUNG**
**2026**

# DAFTAR ISI

DAFTAR ISI.....2
BAB 1
ALGORITMA PENYELESAIAN.....3
1.1. Algorima Divide and Conquer.....3
1.2. Divide and Conquer dalam Voxelization Objek 3D menggunakan Octree..... 3
BAB 2
SOURCE CODE.....6
2.1 Struktur Kode.....6
2.2 Parsing Input dan Output..... 7
2.3 Geometry.....8
2.4 Algoritma Utama..... 10
2.5 Implementasi Bonus Viewer..... 12
2.6 Program Utama..... 15
BAB 3
TESTING.....16
3.1 Test Case 1..... 16
3.2 Test Case 2..... 17
3.3 Test Case 3..... 19
3.4 Test Case 4..... 21
3.5 Test Case 5..... 21
3.6 Test Case 6..... 23
LAMPIRAN.....26

# BAB 1
# ALGORITMA PENYELESAIAN

## 1.1. Algorima *Divide and Conquer*

Algoritma Divide and Conquer merupakan strategi algoritma yang bekerja dengan memecah sebuah persoalan yang besar menjadi beberapa upa-persoalan (sub-problems) yang memiliki karakteristik yang sama dengan persoalan semula, tetapi dengan ukuran yang lebih kecil dan idealnya hampir sama besar. Strategi ini mengadopsi prinsip militer kuno, yaitu divide ut impera. Secara umum, algoritma ini dilakukan melalui tiga tahapan utama yaitu Divide atau membagi persoalan, Conquer atau menyelesaikan upa-persoalan secara mandiri maupun secara rekursif, dan Combine atau menggabungkan solusi dari setiap *subproblems* menjadi solusi utuh bagi persoalan awal. Karena *subproblems* yang dihasilkan memiliki karakteristik yang identik dengan masalah aslinya, strategi ini biasa diimplementasikan menggunakan skema rekursif.

Keunggulan utama dari pendekatan ini adalah kemampuannya untuk meningkatkan efisiensi komputasi, seperti pada algoritma pengurutan yang mampu menurunkan kompleksitas waktu dari brute force $O(n^2)$ menjadi $O(n\,log_n)$. Terdapat dua variasi pendekatan utama dalam strategi ini. Pendekatan easy split/hard join seperti pada Merge Sort yang pembagiannya mudah namun penggabungannya sulit secara komputasi, serta pendekatan hard split/easy join seperti pada Quick Sort yang pembagiannya didasarkan pada nilai elemen sehingga penggabungannya menjadi jauh lebih mudah

## 1.2. *Divide and Conquer* dalam Voxelization Objek 3D menggunakan Octree

Konsep Divide and Conquer dalam implementasi ini :

a. Divide : Ruang awal yang membungkus objek dibentuk sebagai kubus akar yang kemudian dibagi menjadi delapan sub kubus yang berukuran sama menggunakan struktur *octree*. Setiap pembagian, node memiliki paling banyak delapan anak.
b. Conquer : Untuk setiap subkubus hasil pembagian, pengecekan dilakukan untuk memeriksa apakah kubus masih beririsan dengan permukaan dari objek 3D. Jika masih beririsan dan *maxdepth* belum tercapai, subkubus dibagi lagi menjadi delapan bagian. Proses ini dilakukan secara rekursif pada tiap anak.
c. Combine : Solusi akhir merupakan gabungan dari seluruh voxel daun yang lolos proses seleksi pada tiap rekursi atau daun yang dalam kedalaman maksimum yang masih beririsan dengan permukaan objek.

Langkah-langkah implementasi algoritma

1. Program membaca file input .obj dimana hanya dua jenis baris yang diproses yaitu vertex (v x y z ) dan face segitiga (f i j k). Setelah dibaca, data disimpan dalam struktur Mesh yang berisi daftar vertex dan segitiga. Setiap segitiga kemudian direpresentasikan menggunakan titik koordinat 3D agar digunakan pada proses selanjutnya, terutama untuk pengecekan irisan dengan kubus.

2. Dilakukan komputasi *Bounding Cube*. Batas minimum dan maksimum dari seluruh vertex objek dihitung sebelum membangun octree. Dari dua titik batas ini *bounding box* dibentuk dan kemudian diperluas menjadi kubus menggunakan fungsi AABB::fromBounds(...). Proses ini dilakukan untuk memastikan bahwa seluruh objek ada di dalam satu kubus akar karena *octree* membagi ruang dalam bentuk kubus menjadi delapan bagian yang sama besar. Ditambahkan padding agar vertex yang berada di tepi bounding cube tercakup dan hitung titik tengah agar kubus sebagai rata-rata titik minimum dan maksimum.
3. Pembangunan octree dilakukan pada fungsi buildRecursive(node, triangles, maxDepth) dimana tiap pemanggilan rekursif merepresentasikan satu tingkat pembagian ruang. Proses pada setiap node adalah:
    - Membangun Octree dimulai dengan mencatat node yang terbentuk agar tahu node yang terbentuk pada tiap kedalaman pohon.
    - Lalu, dilakukan filter segitiga kandidat. Setiap node menyimpan sebuah kubus (AABB) dan pengecekan irisan dengan seluruh segitiga kandidat yang diwariskan node induk dilakukan terhadap kubus. Hasil dari tahap ini adalah himpunan segitiga yang masih mungkin ada dalam ruang node tersebut.
    - Jika setelah penyaringan tidak ada segitiga yang beririsan dengan kubus, maka node dinyatakan kosong. State node diatur menjadi EMPTY, nodeSkippedPerDepth ditambah, dan rekursi dihentikan untuk node tersebut.
    - Jika node masih beririsan dengan permukaan objek dan kedalaman node sama dengan kedalaman maksimum, node tersebut dinyatakan sebagai VOXEL_LEAF. Node ini representasi satu voxel akhir. Ini memastikan semua voxel akhir memiliki ukuran yang sama.
    - Jika node masih beririsan dengan permukaan objek tetapi kedalaman maksimum belum tercapai, maka node dibagi menjadi delapan subkubus dan fungsi buildRecursive(...) kembali dipanggil pada tiap anak.
4. Dilakukan Uji Interseksi Segitiga-AABB (Separating Axis Theorem) dimana dua objek konveks tidak bersinggungan jika dan hanya jika terdapat setidaknya satu sumbu yang memisahkan proyeksi keduanya. Jika tidak ada sumbu pemisah yang ditemukan dari semua kandidat sumbu, maka objek pasti bersinggungan. Tiga kelompok sumbu yang diuji adalah : 3 sumbu normal face AABB, 1 sumbu normal face segitiga, dan 9 sumbu cross product tepi.
5. Proses subdivisi AABB menjadi 8 oktant dilakukan. Saat sebuah node Internal dibagi, box dibagi menjadi 8 oktant yang sama besar. Jika halfSideLength node parent adalah h, maka setiap anak memiliki halfSideLength = h/2. Pusat setiap anak diperoleh dengan menggeser pusat parent sebesar h/2 ke setiap kombinasi arah ±x, ±y, ±z.
6. Paralelisasi dengan Concurrency. Untuk mempercepat pembangunan Octree, 8 subproblem dijalankan secara paralel pada level teratas pohon hingga kedalaman parallelLevel.

7. Setelah seluruh octree selesai dibangun, program melakukan traversal untuk mengumpulkan semua node VOXEL_LEAF. Setiap node daun berisi sebuah AABB yang merepresentasikan posisi dan ukuran voxel. Kemudian dikonversi menjadi mesh kubus berisi 12 segitiga dan 8 titik sudut lalu ditulis ke file *output* .obj.

# BAB 2
# SOURCE CODE

## 2.1 Struktur Kode

```
Tucil2_13524098_13524131/
├── src/
│   ├── main.cpp
│   │
│   ├── io/
│   │   ├── objParser.cpp
│   │   ├── objParser.hpp
│   │   ├── objWriter.cpp
│   │   └── objWriter.hpp
│   │
│   ├── geometry/
│   │   ├── Vector3D.hpp
│   │   ├── Triangle.hpp
│   │   ├── AABB.hpp
│   │   ├── Mesh.hpp
│   │   ├── Mesh.cpp
│   │   ├── Intersection.hpp
│   │   └── Intersection.cpp
│   │
│   ├── octree/
│   │   ├── Octree.hpp
│   │   ├── Octree.cpp
│   │   └── octreeNode.hpp
│   │
│   ├── voxel/
│   │   ├── voxelizer.cpp # main voxelization logic
│   │   └── voxelizer.hpp
│   │
│   ├── viewer/
│
├── test/
├── doc/
│   └── LaporanTucil2_13524098_13524131.pdf
│
└── README.md
```

## 2.2 Parsing Input dan Output

**objParser.cpp :**

File ini menangani berbagai format face dan triangulasi otomatis. Semua token yang bukan ‘v’ atau ‘f’ diabaikan, parseFaceVertex() mengambil hanya indeks vertex (sebelum '/').

```cpp
    if (prefix == "v") {
        Vector3D vertex;
        if (!parseVertexLine(line, vertex)) {
            result.errorMessage = "Invalid vertex at line " + std::to_string(lineNumber);
            return result;
        }
        mesh.vertices.push_back(vertex);
    }
    else if (prefix == "f") {
        int i, j, k;
        if (!parseFaceLine(line, i, j, k)) {
            result.errorMessage = "Invalid face at line " + std::to_string(lineNumber);
            return result;
        }

        if (i < 1 || j < 1 || k < 1 ||
            i > (int)mesh.vertices.size() ||
            j > (int)mesh.vertices.size() ||
            k > (int)mesh.vertices.size()) {
            result.errorMessage = "Face index is out of range at line " + std::to_string(lineNumber);
            return result;
        }

        mesh.triangles.emplace_back(
            mesh.vertices[i - 1],
            mesh.vertices[j - 1],
            mesh.vertices[k - 1]
        );
    }
}
```

**objWriter.cpp :**

Setiap triangle ditulis sebagai 3 baris vertex baru sehingga setiap face selalu f N N+1 N+2. printStats(...) menampilkan jumlah voxel, vertex, face, dan menampilkan statistik node octree yang terbentuk.

```cpp
int currentIndex = 1;

for (const Triangle& tri : mesh.triangles) {
    out << "v " << tri.v0.x << " " << tri.v0.y << " " << tri.v0.z << "\n";
    out << "v " << tri.v1.x << " " << tri.v1.y << " " << tri.v1.z << "\n";
    out << "v " << tri.v2.x << " " << tri.v2.y << " " << tri.v2.z << "\n";

    out << "f " << currentIndex << " " << currentIndex + 1 << " " << currentIndex + 2 << "\n";
    currentIndex += 3;
}
```

## 2.3 Geometry

**AABB.hpp :**

Merepresentasikan kubus atau kotak pembatas (bounding box). fromBounds() membuat bounding box/cube yang selalu berbentuk kubus dengan padding agar vertex di tepi masih tercakup. subdivide() membagi 1 kubus menjadi 8 oktant dengan menggeser pusat +/- halfside/2 ke semua kombinasi arah.

```
static AABB fromBounds(Vector3D minPoint, Vector3D maxPoint) {
        double lengthX = maxPoint.x - minPoint.x;
        double lengthY = maxPoint.y - minPoint.y;
        double lengthZ = maxPoint.z - minPoint.z;

        double longestSide = lengthX;
        if (lengthY > longestSide) {
            longestSide = lengthY;
        }
        if (lengthZ > longestSide) {
            longestSide = lengthZ;
        }

        // Padding supaya titik yang pas di pinggiran pun bisa tetep ke-include
        double halfSideLength = (longestSide / 2.0) * 1.001;

        Vector3D centerPoint(
            (minPoint.x + maxPoint.x) / 2.0,
            (minPoint.y + maxPoint.y) / 2.0,
            (minPoint.z + maxPoint.z) / 2.0
        );

        return AABB(centerPoint, halfSideLength);
    }

    // Bagi AABB menjadi 8 octant (anak) yang lebih kecil
    void subdivide(AABB children[8]) const {
        double childHalfSideLength = halfSideLength / 2.0;

        double centerX = center.x;
        double centerY = center.y;
        double centerZ = center.z;

        // 8 kombinasi arah vektor dari titik pusat parent
        children[0] = AABB(Vector3D(centerX - childHalfSideLength, centerY - childHalfSideLength, centerZ - childHalfSideLength), childHalfSideLength);
        children[1] = AABB(Vector3D(centerX - childHalfSideLength, centerY - childHalfSideLength, centerZ + childHalfSideLength), childHalfSideLength);
        children[2] = AABB(Vector3D(centerX - childHalfSideLength, centerY + childHalfSideLength, centerZ - childHalfSideLength), childHalfSideLength);
        children[3] = AABB(Vector3D(centerX - childHalfSideLength, centerY + childHalfSideLength, centerZ + childHalfSideLength), childHalfSideLength);
        children[4] = AABB(Vector3D(centerX + childHalfSideLength, centerY - childHalfSideLength, centerZ - childHalfSideLength), childHalfSideLength);
        children[5] = AABB(Vector3D(centerX + childHalfSideLength, centerY - childHalfSideLength, centerZ + childHalfSideLength), childHalfSideLength);
        children[6] = AABB(Vector3D(centerX + childHalfSideLength, centerY + childHalfSideLength, centerZ - childHalfSideLength), childHalfSideLength);
        children[7] = AABB(Vector3D(centerX + childHalfSideLength, centerY + childHalfSideLength, centerZ + childHalfSideLength), childHalfSideLength);
    }
```

**Intersection.cpp :**

File ini menangani pengecekan irisan geometri, antara segitiga dan kubus. Di fungsi ini dilakukan translasi ke koordinat lokal box dan mengelompokkan menjadi 3 grup. Grup pertama adalah 3 sumbu normal box dimana jika proyeksi segitiga di luar [-h, +h] maka terpisah. Grup kedua adalah 1 sumbu normal segitiga dan grup ketiga adalah 9 cross product dari 3 tepi box x 3 tepi segitiga .

```cpp
bool triangleIntersectsAABB(const Triangle& tri, const AABB& box)
{
    Vector3D c = box.center;
    double   h = box.halfSideLength;

    Vector3D v0 = tri.v0 - c;
    Vector3D v1 = tri.v1 - c;
    Vector3D v2 = tri.v2 - c;

    {
        double mn, mx;

        mn = std::min({v0.x, v1.x, v2.x});
        mx = std::max({v0.x, v1.x, v2.x});
        if (mn > h || mx < -h) return false;

        mn = std::min({v0.y, v1.y, v2.y});
        mx = std::max({v0.y, v1.y, v2.y});
        if (mn > h || mx < -h) return false;
        mn = std::min({v0.z, v1.z, v2.z});
        mx = std::max({v0.z, v1.z, v2.z});
        if (mn > h || mx < -h) return false;
    }

    {
        Vector3D e0 = v1 - v0;
        Vector3D e1 = v2 - v0;
        Vector3D normal = e0.cross(e1);

        double d  = normal.dot(v0);
        double r  = aabbRadius(normal, h, h, h);
        double absD = std::fabs(d);
        if (absD > r) return false;
    }

    Vector3D triEdges[3] = { v1 - v0, v2 - v1, v0 - v2 };

    for (int i = 0; i < 3; ++i) {
        const Vector3D& e = triEdges[i];
        {
            Vector3D axis(0.0, -e.z, e.y);
            double triMin, triMax;
            projectTriangle(axis, v0, v1, v2, triMin, triMax);
            double r = aabbRadius(axis, h, h, h);
            if (!overlaps(triMin, triMax, 0.0, r)) return false;
        }

        {
            Vector3D axis(e.z, 0.0, -e.x);
            double triMin, triMax;
            projectTriangle(axis, v0, v1, v2, triMin, triMax);
            double r = aabbRadius(axis, h, h, h);
            if (!overlaps(triMin, triMax, 0.0, r)) return false;
        }

        {
            Vector3D axis(-e.y, e.x, 0.0);
            double triMin, triMax;
            projectTriangle(axis, v0, v1, v2, triMin, triMax);
            double r = aabbRadius(axis, h, h, h);
            if (!overlaps(triMin, triMax, 0.0, r)) return false;
        }
    }
    return true;
}
```

## 2.4 Algoritma Utama

Inti dari algoritma *Divide and Conquer* ada dalam Octree. Logika paling penting berada di buildRecursive(...). Fungsi ini mencatat node terbentuk dan filter yang beririsan dengan box node jika kosong maka prune cabang, jika mencapai maxDepth maka jadi voxel leaf, dan jika belum maksimum maka dibagi jadi 8 anak dan proses rekursif dilakukan kembali.

Dalam Octree.cpp bagian **bonus concurrency** diimplementasikan pada pemrosesan anak-anak node. Delapan subproblem anak secara paralel pada level teratas pohon dijalankan. Pada bagian if (node->depth <= parallelLevel) dilakukan filter segitiga per anak untuk menghindari data race. Lalu dijalankan delapan subproblems secara asinkron lalu ditunggu hingga semua selesai.

```cpp
bool Octree::buildRecursive(OctreeNode* node,
                            const std::vector<const Triangle*>& triangles,
                            int maxDepth)
{
    incrementNodeCount(node->depth);

    std::vector<const Triangle*> intersecting;
    intersecting.reserve(triangles.size());
    for (const Triangle* tri : triangles) {
        if (Intersection::triangleIntersectsAABB(*tri, node->box))
            intersecting.push_back(tri);
    }

    if (intersecting.empty()) {
        node->state = NodeState::Empty;
        incrementSkippedCount(node->depth);
        return false;
    }

    if (node->depth >= maxDepth) {
        node->state = NodeState::VoxelLeaf;
        incrementVoxelCount();
        return true;
    }

    node->state = NodeState::Internal;
    AABB childBoxes[8];
    node->box.subdivide(childBoxes);
    bool hasAnyVoxel = false;

    if (node->depth <= parallelLevel) {
        std::vector<std::vector<const Triangle*>> childTris(8);
        for (int i = 0; i < 8; ++i) {
            for (const Triangle* tri : intersecting) {
                if (Intersection::triangleIntersectsAABB(*tri, childBoxes[i]))
                    childTris[i].push_back(tri);
            }
        }

        std::future<bool> futures[8];
        for (int i = 0; i < 8; ++i) {
            node->children[i] = std::make_unique<OctreeNode>(childBoxes[i], node->depth + 1);
            OctreeNode* childPtr = node->children[i].get();
            auto localTris = std::move(childTris[i]);
            futures[i] = std::async(std::launch::async,
                [this, childPtr, tris = std::move(localTris), maxDepth]() mutable -> bool {
                    return buildRecursive(childPtr, tris, maxDepth);
                });
        }

        for (int i = 0; i < 8; ++i) {
            bool childHasVoxel = futures[i].get();
            if (!childHasVoxel)
                node->children[i].reset();
            else
                hasAnyVoxel = true;
        }
    } else {
        for (int i = 0; i < 8; ++i) {
            node->children[i] = std::make_unique<OctreeNode>(childBoxes[i], node->depth + 1);
            bool childHasVoxel = buildRecursive(node->children[i].get(), intersecting, maxDepth);
            if (!childHasVoxel)
                node->children[i].reset();
            else
                hasAnyVoxel = true;
        }
    }

    return hasAnyVoxel;
}
```

**Voxelizer.cpp** digunakan untuk mengikat atau menghubungkan semua octree dengan hasil akhir. Octree dibangun dari mesh input, lalu semua voxel leaf dikumpulkan ke daftar AABB, setiap voxel dikonversi menjadi mesh kubus (12 segitiga), dan semua digabung ke outputMesh .

```
Octree octree;
octree.build(inputMesh, maxDepth, parallelDepthLimit);

std::vector<AABB> voxelBoxes;
octree.collectVoxels(voxelBoxes);

Mesh outputMesh;
for (const AABB& box : voxelBoxes) {
    Mesh cubeMesh = Mesh::createCubeMesh(box);
    outputMesh.append(cubeMesh);
}
```

## 2.5 Implementasi Bonus Viewer

Implementasi bonus viewer terletak di file **Viewer.cpp**. Di dalamnya, terdapat beberapa algoritma dan konsep teknis yang diaplikasikan.

**Backface culling :**

Setiap face memiliki normal (vektor tegak lurus permukaan yang mengarah ke luar). Dot product antara normal face dan arah kamera menghasilkan nilai positif jika face menghadap kamera, dan negatif jika membelakangi kamera. Face yang membelakangi kamera tidak digambar, sehingga hanya permukaan luar objek yang terlihat.

```
double facingCamera = face.normal.dot(cameraDirection);
if (facingCamera <= 0.0) {
    continue;
}
```

**Depth sorting (painter's algorithm) :**

Painter's algorithm menggambar face dari yang paling jauh ke yang paling dekat ke kamera, sehingga face yang dekat secara otomatis menutupi face yang jauh. Depth setiap face dihitung sebagai rata-rata nilai Z keempat titik sudutnya di camera space.

```
sort(readyFaces.begin(), readyFaces.end(),
    [](const ReadyFace& faceA, const ReadyFace& faceB)
{       return faceA.depth > faceB.depth;
    }
);
```

**Perspective projection :**

Proyeksi perspektif membagi koordinat X dan Y dengan nilai Z di camera space, sehingga objek yang jauh terlihat lebih kecil. focalLength mengontrol lebar sudut pandang kamera. Hasil proyeksi kemudian dipetakan dari rentang -1..1 ke koordinat pixel layar. Sumbu Y dibalik karena di layar Y bertambah ke bawah, sedangkan di koordinat 3D Y bertambah ke atas.

```
double projectedX = cameraX * camera.focalLength / cameraZ;
double projectedY = cameraY * camera.focalLength / cameraZ;

double screenX = (projectedX + 1.0) / 2.0 * (double)WINDOW_WIDTH;
double screenY = (1.0 - projectedY) / 2.0 * (double)WINDOW_HEIGHT;
```

**View matrix :**

View matrix mengubah koordinat dari world space ke camera space — seolah-olah kamera ada di origin dan menghadap ke arah -Z. Tiga sumbu kamera dihitung dari posisi kamera (eye), titik yang dilihat (target), dan arah atas (up). Kolom keempat menyimpan translasi yang menggeser seluruh dunia sehingga kamera berada di origin.

```
Vector3D forward     = (camera.target -
Vector3D right       = forward.cross(camera.up).normalize();
Vector3D correctedUp = right.cross(forward);

viewMatrix.data[0][3] = -(right.x * camera.eye.x +
                          right.y * camera.eye.y +
                          right.z * camera.eye.z);
```

**Spherical coordinates untuk orbit kamera :**

Posisi kamera disimpan dalam spherical coordinates (radius, theta, phi) bukan Cartesian (x, y, z). Saat user drag mouse, hanya theta dan phi yang diubah, lalu posisi kamera dihitung ulang dari rumus ini. Cara ini membuat kamera selalu mengorbit mengelilingi titik tengah objek dengan jarak tetap, tanpa perlu menghitung rotasi matrix secara manual.

```
camera.eye.x = camera.target.x + camera.orbitRadius * sinPhi * cosTheta;
camera.eye.y = camera.target.y + camera.orbitRadius * cosPhi;
camera.eye.z = camera.target.z + camera.orbitRadius * sinPhi * sinTheta;
```

**Diffuse lighting :**

Kecerahan setiap face dihitung dari dot product antara normal face dan arah cahaya. Semakin tegak lurus face terhadap arah cahaya, semakin terang. Nilai 0.4 adalah ambient light (cahaya dasar yang selalu ada) dan 0.6 adalah diffuse light (cahaya yang bergantung sudut). Arah cahaya mengikuti posisi kamera supaya objek selalu terlihat jelas dari semua sudut pandang.

```
Vector3D toCamera       = (camera.eye - camera.target).normalize();
Vector3D lightDirection = (toCamera + lightUp * 0.5).normalize();

double lightDot        = face.normal.dot(lightDirection);
ready.brightness       = 0.4 + 0.6 * lightDot;
```

## 2.6 Program Utama

**main.cpp** mengatur alur program secara keseluruhan. File ini membaca argumen input, validasi dile dan maxDepth, memanggil parser dan voelizer, menyimpan hasil file ke .obj, dan print statistik CLI.

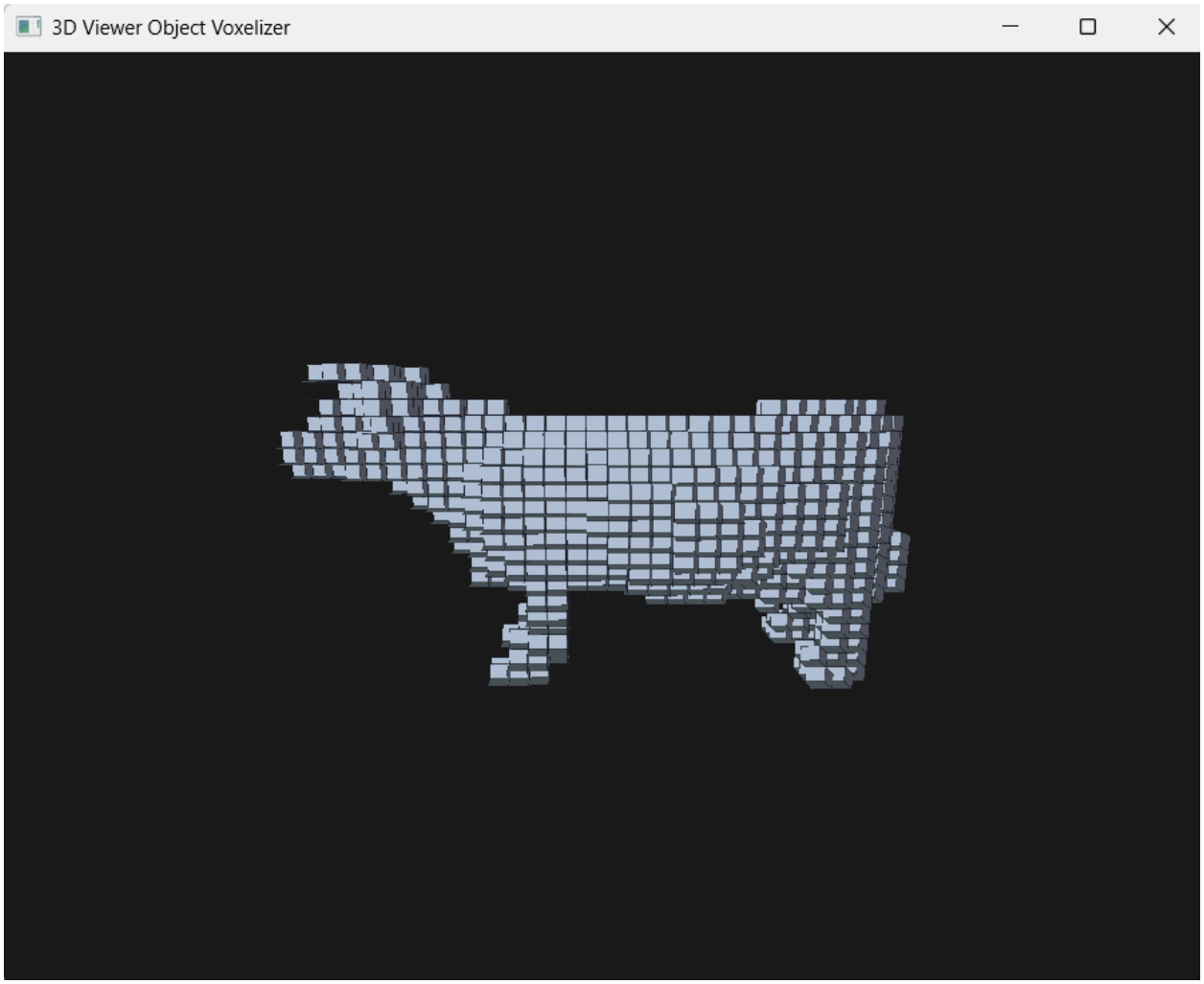
# BAB 3
# TESTING

## 3.1 Test Case 1

Input file: test/cow.obj

Run command: ./voxelizer ../test/cow.obj 6 ../test/cow-depth6.obj 2 --show

Output:



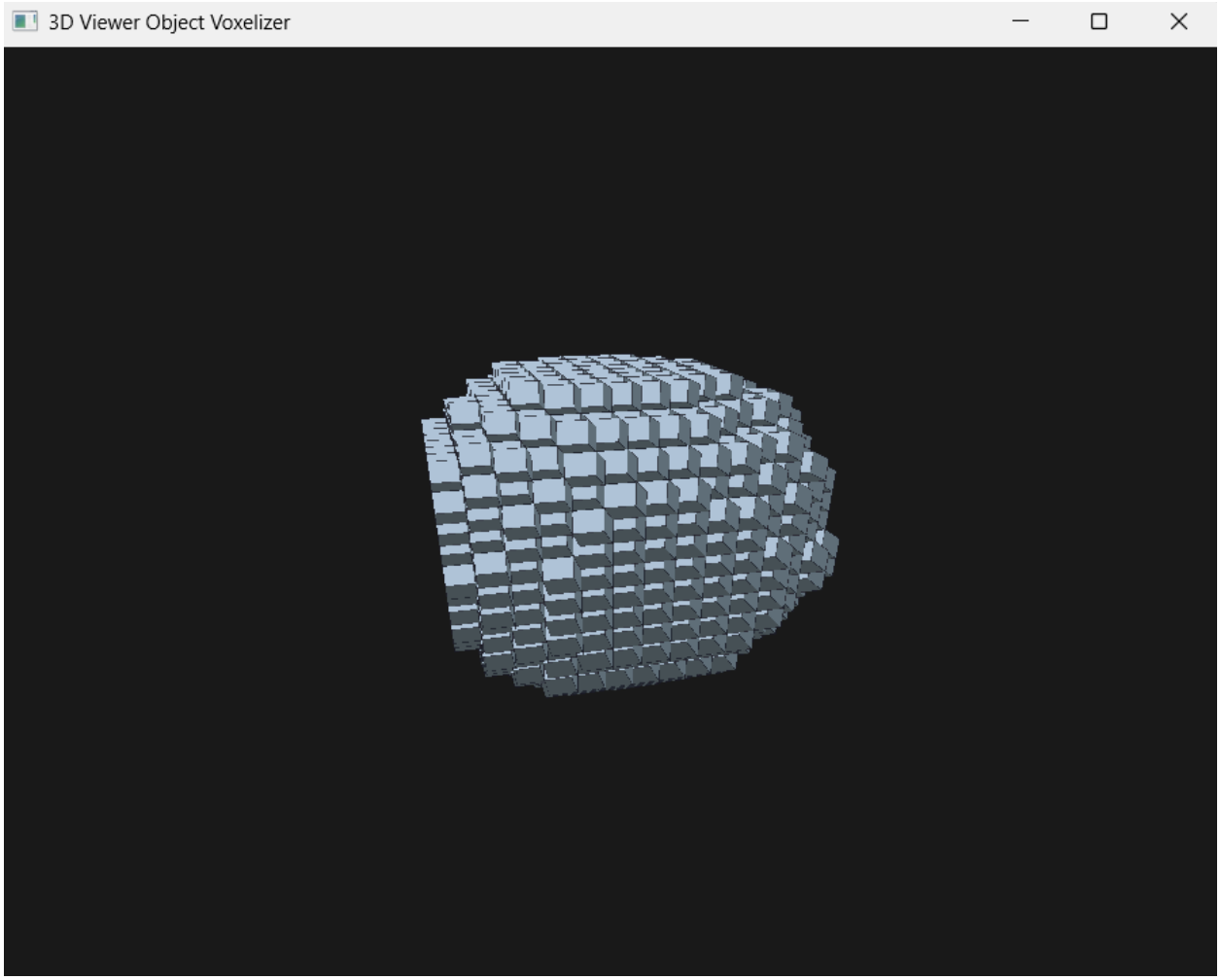
```
Parsing: ../test/cow.obj ...
  Vertices: 4583  Triangles: 5804
Voxelizing (maxDepth=6, parallelLevel=2) ...
Opening Viewer... (Close window to see stats)

=== Hasil ===
Banyaknya voxel yang terbentuk: 1362
Banyaknya vertex yang terbentuk: 10896
Banyaknya faces yang terbentuk: 16344

Statistik node octree yang terbentuk:
1 : 1
2 : 8
3 : 64
4 : 160
5 : 704
6 : 2656

Statistik node yang tidak perlu ditelusuri:
1 : 0
2 : 0
3 : 44
4 : 72
5 : 372
6 : 1294

Kedalaman octree: 6
Lama waktu program berjalan: 36.471 ms
Path dimana file .obj disimpan: ../test\../test/cow-depth6.obj
```

### 3.2 Test Case 2

Input file: test/pumpkin.obj

Run command: ./voxelizer ../test/pumpkin.obj 5 ../test/pumpkin-depth5.obj 2 --show

Output:


3D Viewer Object Voxelizer

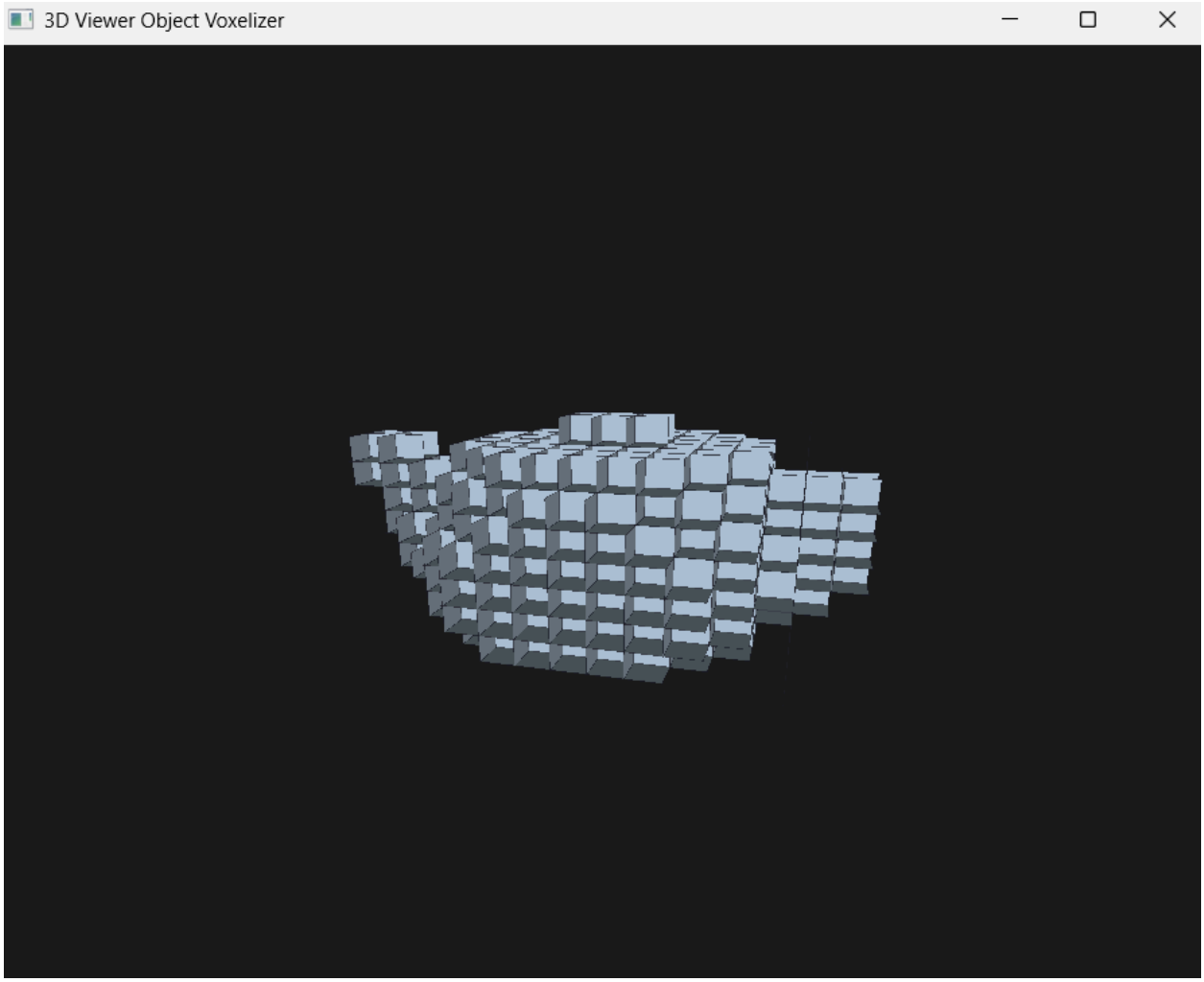
```
Parsing: ../test/pumpkin.obj ...
  Vertices: 5002  Triangles: 10000
Voxelizing (maxDepth=5, parallelLevel=2) ...
Opening Viewer... (Close window to see stats)

=== Hasil ===
Banyaknya voxel yang terbentuk: 1121
Banyaknya vertex yang terbentuk: 8968
Banyaknya faces yang terbentuk: 13452

Statistik node octree yang terbentuk:
1 : 1
2 : 8
3 : 64
4 : 448
5 : 2216

Statistik node yang tidak perlu ditelusuri:
1 : 0
2 : 0
3 : 8
4 : 171
5 : 1095

Kedalaman octree: 5
Lama waktu program berjalan: 43.388 ms
Path dimana file .obj disimpan: ../test\../test/pumpkin-depth5.obj
```

### 3.3 Test Case 3

Input file: test/teapot.obj

Run command: ./voxelizer ../test/teapot.obj 5 ../test/teapot-depth5.obj 2 --show

Output:


3D Viewer Object Voxelizer

```
Parsing: ../test/teapot.obj ...
  Vertices: 530  Triangles: 1024
Voxelizing (maxDepth=5, parallelLevel=2) ...
Opening Viewer... (Close window to see stats)

=== Hasil ===
Banyaknya voxel yang terbentuk: 418
Banyaknya vertex yang terbentuk: 3344
Banyaknya faces yang terbentuk: 5016

Statistik node octree yang terbentuk:
1 : 1
2 : 8
3 : 64
4 : 192
5 : 864

Statistik node yang tidak perlu ditelusuri:
1 : 0
2 : 0
3 : 40
4 : 84
5 : 446

Kedalaman octree: 5
Lama waktu program berjalan: 27.427 ms
Path dimana file .obj disimpan: ../test\../test/teapot-depth5.obj
```

### 3.4 Test Case 4

Input file: test/tidakada.obj

Run command: ./voxelizer ../test/tidakada.obj 5

Output:

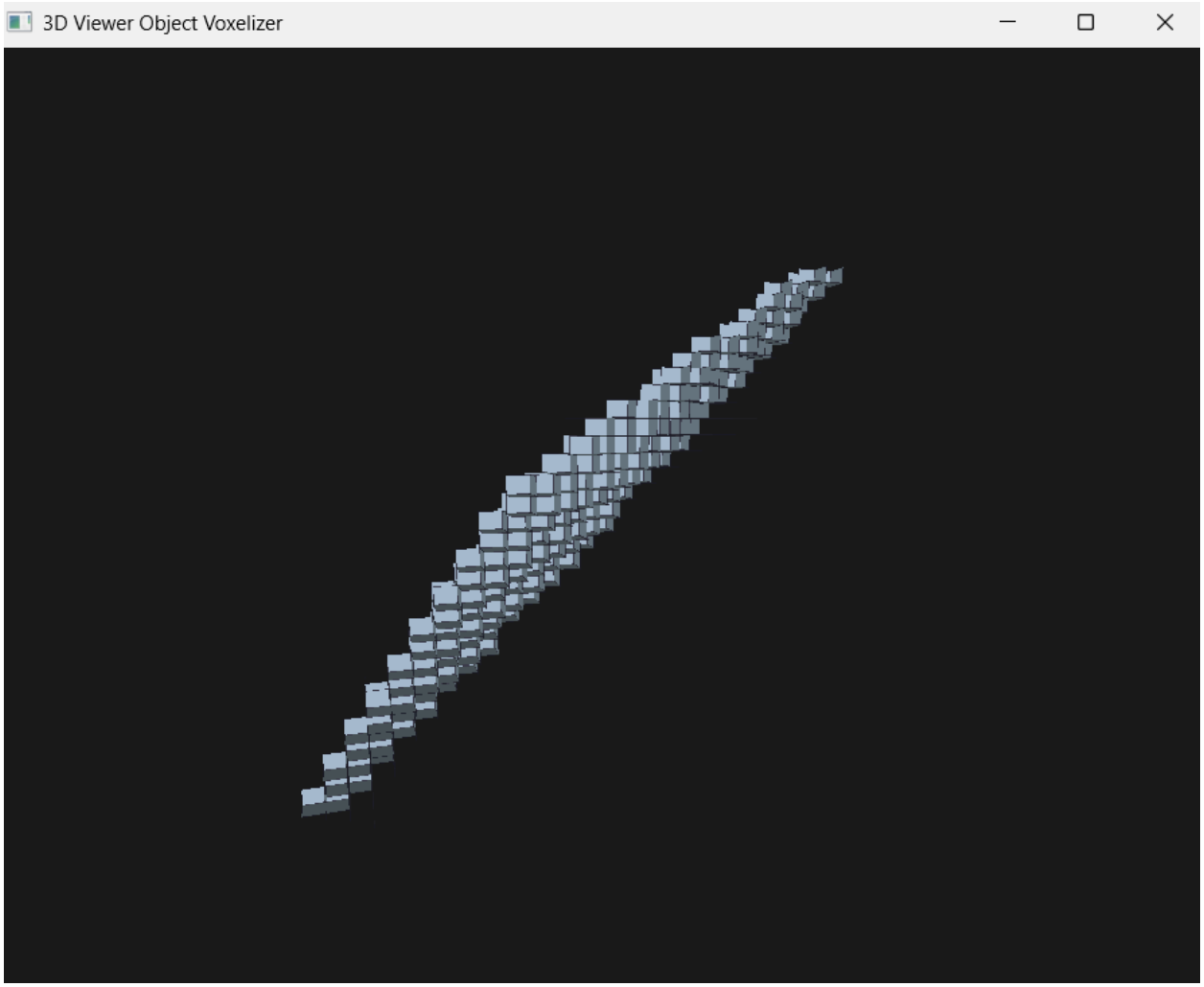
```
PS E:\Kuliah\Mata Kuliah\Semester 4\Strategi Algoritma\Tucil 2\Tucil2_13524098_13524131\src> ./voxelizer ../test/tidakada.obj 5
Error: input file does not exist: ../test/tidakada.obj
```

### 3.5 Test Case 5

Input file: test/line.obj

Run command: ./voxelizer ../test/line.obj 6 ../test/line-depth6.obj 2 --show

Output:


3D Viewer Object Voxelizer

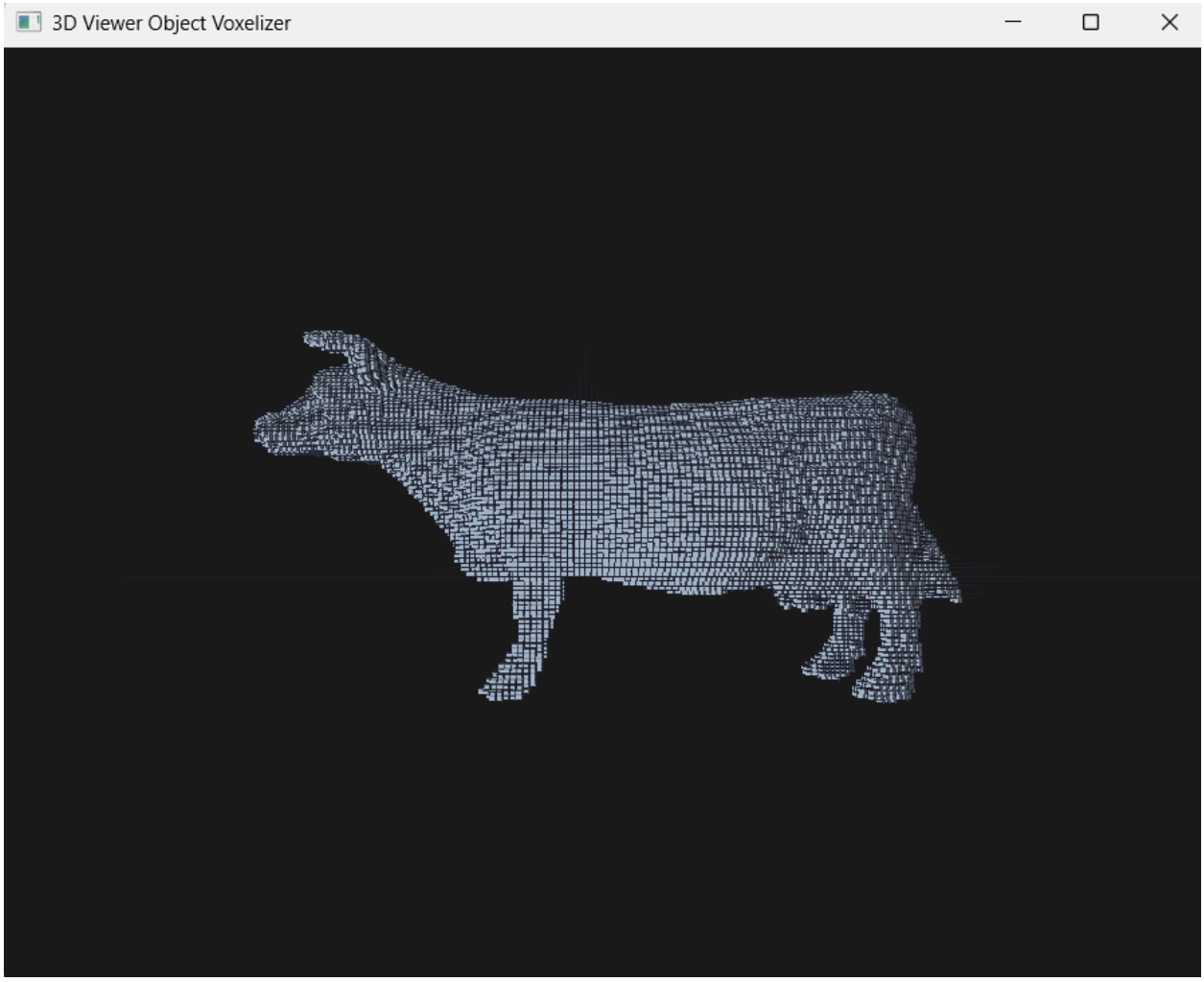
```
Parsing: ../test/line.obj ...
  Vertices: 3  Triangles: 1
Voxelizing (maxDepth=6, parallelLevel=2) ...
Opening Viewer... (Close window to see stats)

=== Hasil ===
Banyaknya voxel yang terbentuk: 287
Banyaknya vertex yang terbentuk: 2296
Banyaknya faces yang terbentuk: 3444

Statistik node octree yang terbentuk:
1 : 1
2 : 8
3 : 56
4 : 120
5 : 264
6 : 744

Statistik node yang tidak perlu ditelusuri:
1 : 0
2 : 1
3 : 41
4 : 87
5 : 171
6 : 457

Kedalaman octree: 6
Lama waktu program berjalan: 22.529 ms
Path dimana file .obj disimpan: ../test\../test/line-depth6.obj
```

### 3.6 Test Case 6

Input file: test/cow.obj

Run command: ./voxelizer ../test/cow.obj 8 ../test/cow-depth8.obj 4 --show

Output:


3D Viewer Object Voxelizer

```
Parsing: ../test/cow.obj ...
  Vertices: 4583  Triangles: 5804
Voxelizing (maxDepth=8, parallelLevel=4) ...
Opening Viewer... (Close window to see stats)

=== Hasil ===
Banyaknya voxel yang terbentuk: 22798
Banyaknya vertex yang terbentuk: 182384
Banyaknya faces yang terbentuk: 273576

Statistik node octree yang terbentuk:
1 : 1
2 : 8
3 : 64
4 : 160
5 : 704
6 : 2656
7 : 10896
8 : 44864

Statistik node yang tidak perlu ditelusuri:
1 : 0
2 : 0
3 : 44
4 : 72
5 : 372
6 : 1294
7 : 5288
8 : 22066

Kedalaman octree: 8
Lama waktu program berjalan: 358.081 ms
Path dimana file .obj disimpan: ../test\../test/cow-depth8.obj
```

# LAMPIRAN

**Link Repository**

https://github.com/revanatania/Tucil2_13524098_13524131

**Tabel Checklist**

| No | Poin | Ya | Tidak |
|---|---|---|---|
| 1 | Program berhasil dikompilasi tanpa kesalahan | ✓ | |
| 2 | Program berhasil dijalankan | ✓ | |
| 3 | Hasil konversi .obj terdiri dari kubus-kubus kecil | ✓ | |
| 4 | Program informasi-informasi atau report | ✓ | |
| 5 | Program dapat menerima masukan .obj, dan mengembalikan output | ✓ | |
| 6 | [Bonus] Program menggunakan concurrency | ✓ | |
| 7 | [Bonus] Program dapat menampilkan visualisasi .obj | ✓ | |

**Pernyataan**

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan *(Generative AI)*, melainkan hasil pemikiran dan analisis mandiri.

Reva Natania Sitohang

Amanda Aurellia Salsabilla